FOR COMPLETE BEGINNERS

Protect your object's data.

Complete Modern Java, episode 17: encapsulation, in Java and TypeScript

> Private fields, getters, and a setter that refuses a negative price.

With open fields, bread.priceCents = -5 runs and prints -5. Nothing stops it.

Encapsulation means the object controls its own data: private fields, and methods that check.

In this episode

- A class in its own file
- private fields
- Getters
- A setter that checks
- throw, in one sentence
- The constructor uses the setter
- TypeScript private

Coming next

- Exceptions: try and catch (episode 18)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

Fields.java (open fields): bread.priceCents = -5	`-5`
Store.java: bread.priceCents = -5 (private)	`error: priceCents has private access in Product`
Getters	`Bread, 150`
setPriceCents(-500), no check	`-500`
setPriceCents(-5), with the check	`IllegalArgumentException: price cannot be negative`
product.ts: bread.priceCents	`tsc TS2341; node still prints 150`

Every row was run, not predicted.

02 Private fields, doors that check

private: only code inside Product can use the field. Getters read it; the setter checks before it changes it.

The constructor calls the same setter, so the rule lives in one place. name has no setter: it cannot change.

Product.java

```
class Product {
    private String name;
    private int priceCents;

    Product(String name, int priceCents) {
        this.name = name;
        setPriceCents(priceCents);
    }

    String getName() { return name; }
    int getPriceCents() { return priceCents; }

    void setPriceCents(int newPriceCents) {
        if (newPriceCents < 0) {
            throw new IllegalArgumentException("price cannot be negative");
        }
        priceCents = newPriceCents;
    }
}
```

> Store.java printed Bread, 150, 200, then stopped at new Product("Eggs", -5) with the exception.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 Why Product moved to its own file

Declared in the same compact file as main, Product is nested inside the file's own class, and Java lets that class reach its private members: the same line printed -5.

In its own file, Product.java, main is outside the class and the line is refused.

Store.java

```
void main() {
    Product bread = new Product("Bread", 150);
    bread.priceCents = -5;   // error: priceCents has private access in Product
}
```

> Same.java (one file) printed -5; Store.java with Product.java reported the private access error.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript private

tsc refuses bread.priceCents, but node does not check types and prints 150. TypeScript private is a rule for the checker; JavaScript #private fields are enforced at run time.

product.ts

```
class Product {
    private priceCents: number;
    constructor(priceCents: number) {
        this.priceCents = priceCents;
    }
    getPriceCents(): number {
        return this.priceCents;
    }
}
const bread = new Product(150);
console.log(bread.priceCents);   // tsc: TS2341; node: 150
```

> npx tsc --noEmit reported TS2341; node product.ts printed 150 and 150.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Protect your data

1. Fields private.
2. Getters to read.
3. A setter only if the value may change, and it checks.
4. The constructor uses the same check.
5. Challenge: setPriceCents(300) or priceCents = 300: which compiles?

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.